参考指南

TMS320F28P55x Flash API 4.00.00.00 版

摘要

Flash 是一种可通过电方式擦除和编程的非易失性存储器，可反复进行编程和擦除，以便于代码开发。Flash 存储器主要可用作内核的程序存储器，也可用作静态数据存储器。本指南介绍如何使用 Flash API 库，对 TMS320F28P55x 器件的片上 Flash 存储器执行擦除、编程和验证操作。

目录

1 引言 3
1.1 参考资料 3
1.2 功能列表格式 3
2 TMS320F28P55x Flash API 概述 4
2.1 引言 4
2.2 API 概述 4
2.3 使用 API 5
3 API 函数 8
3.1 初始化函数 8
3.2 Flash 状态机函数 8
3.3 读取函数 24
3.4 信息查询函数 26
3.5 实用工具函数 26
4 建议的 FSM 流程 28
4.1 出厂新器件 28
4.2 建议的擦除流程 29
4.3 建议的 Bank 擦除流程 29
4.4 建议的编程流程 31
5 与安全相关的软件应用使用假设 32
A Flash 状态机命令 33
B 类型定义、宏定义、枚举和结构体 34
B.1 类型定义 34
B.2 宏定义 34
B.3 枚举 34
B.4 结构体 35
参考资料 36

图目录

图 4-1. 建议的擦除流程 29
图 4-2. 建议的 Bank 擦除流程 30
图 4-3. 建议的编程流程 31

表目录

表 2-1. 初始化函数汇总 4
表 2-2. Flash 状态机（FSM）函数汇总 4
表 2-3. 读取函数汇总 5
表 2-4. 信息查询函数汇总 5
表 2-5. 实用工具函数汇总 5
表 3-1. 不同编程模式的用途 13
表 3-2. Fapi_issueProgrammingCommand() 的允许编程范围 13
表 3-3. Fapi_issueAutoEcc512ProgrammingCommand() 的允许编程范围 18
表 3-4. Fapi_issueDataAndEcc512ProgrammingCommand() 的允许编程范围 19

Copyright © 2024 Texas Instruments Incorporated

表目录（续）

表 3-5. Fapi_issueDataOnly512ProgrammingCommand() 的允许编程范围 21
表 3-6. 64 位 ECC 数据解释 22
表 3-7. Fapi_issueEccOnly64ProgrammingCommand() 的允许编程范围 22
表 3-8. STATCMD 寄存器 24
表 3-9. STATCMD 寄存器字段说明 24
表 A-1. Flash 状态机命令 33

商标

Arm® 是 Arm Limited（或其子公司）在美国和/或其他地区的注册商标。
所有商标均归其各自所有者所有。

Copyright © 2024 Texas Instruments Incorporated

1 引言

本参考指南详细介绍了 Texas Instruments TMS320F28P55x Flash API 库（FAPI_F28P55x_EABI_v4.00.00.00 和 FAPI_F28P55x_COFF_v4.00.00.00）中可用于擦除、编程和验证 TMS320F28P55x 器件 Flash 的函数。尽管 TMS320F28P55x 的库名称不同，但 Flash API 库内容相同。请注意，Flash API V4.00.XX.XX 仅可用于 TMS320F28P55x 器件。TMS320F28P55x 的 Flash API 库随 C2000Ware 提供，位于 C2000Ware_x_xx_xx_xx\libraries\flash_api\f28p55x。

1.1 参考资料

本指南应与 *TMS320F28P55x Real-Time Microcontrollers Data Manual* 和 *TMS320F28P55x Real-Time Microcontrollers Technical Reference Manual* 配合使用，以了解 TMS320F28P55x 器件。

1.2 函数条目格式

函数、编译器内建函数或宏的条目通常采用以下格式。

简要说明 function_name() 的功能。

函数原型

给出 function_name() 的原型。

```
<return_type> function_name(                <type_1> para
meter_1,                <type_2> parameter_2,                <t
ype_n> parameter_n                        )
```

参数

parameter_1 [in]　　parameter_1 的类型说明　parameter_2 [out]　　paramet
er_2 的类型说明　parameter_n [in/out]　　parameter_3 的类型说明

参数传递分为以下几类：

- in — 表示函数会使用传入参数中的一个或多个值，但不会保存任何更改。
- out — 表示函数会将参数中的一个或多个值保存下来。您可以检查这些已保存的值，以获取有关应用程序的有用信息。
- in/out — 表示函数会更改参数中的一个或多个值，并保存结果。您可以检查这些已保存的值，以获取有关应用程序的有用信息。

说明

介绍该函数。本节还会说明可能适用的特殊特性或限制：

- 函数在某些条件下会阻止或可能阻止所请求的操作
- 函数具有可能不明显的前置条件
- 函数具有使用限制或特殊行为

限制

列出使用此函数时可能存在的限制。

返回值

说明此函数返回的一个或多个值。

Copyright © 2024 Texas Instruments Incorporated

另请参见

列出与此函数相关的其他函数或数据类型。

示例实现

给出一个示例（或示例引用），用于说明该函数的用法。除了 Flash API 函数之外，这些示例还可能使用 C2000Ware 中 device_support 文件夹或 Driverlib 文件夹提供的函数，以展示在应用程序中使用相应 Flash API 函数的方式。

2 TMS320F28P55x Flash API 概述

2.1 引言

Flash API 是一个函数库。按正确顺序并传入正确参数调用其中的例程后，可擦除、编程或验证 Flash 存储器。Flash API 也可用于编程 OTP 存储器。

注

请阅读 *TMS320F28P55x Real-Time Microcontrollers Data Manual*，了解 TMS320F28P55x 的 Flash 和 OTP 存储器映射以及 Flash 等待状态规格。本参考指南假定用户已阅读 *TMS320F28P55x Real-Time Microcontrollers Technical Reference Manual* 中的 *Flash and OTP Memory* 章节。

另请特别注意此器件上的 Fapi_issueAsyncCommand()、Fapi_setupBankSectorEnable() 和 Fapi_issueBankEraseCommand() 函数。这些函数在 TMS320F28P55x 上的用法，可参见 C2000Ware 中提供的 Flash API 用法示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。512-bit 编程中这些函数的用法，可参见 C2000Ware 提供的 Flash API 用法示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。

2.2 API 概述

表 2-1. 初始化函数汇总

API Function	Description
Fapi_initializeAPI()	Initializes the API for first use or frequency change

Table 2-2. Summary of Flash State Machine (FSM) Functions

API Function Description	Description
Fapi_setActiveFlashBank()	Initializes Flash Wrapper and bank for an erase or program command
Fapi_setupBankSectorEnable()	Configures the Write/Erase protection for the sectors.
Fapi_issueBankEraseCommand()	Issues bank erase command to the Flash State Machine for the given bank address.
Fapi_issueAsyncCommandWithAddress()	Issues an erase sector command to FSM for the given address
Fapi_issueProgrammingCommand()	Sets up the required registers for programming and issues the command to the FSM Note: Maximum 128-bits (8 16-bit words) can be programed at once.
Fapi_issueProgrammingCommandForEccAddress()	Remaps an ECC address to the main data space and then call Fapi_issueProgrammingCommand() to program ECC
Fapi_issueAutoEcc512ProgrammingCommand()	Sets up the required registers for 512-bit (32 16-bit words) programming with AutoECC generation mode and issues the command to the FSM
Fapi_issueDataAndEcc512ProgrammingCommand()	Sets up the required registers for 512-bit (32 16-bit words) programming with user provided flash data and ECC, and issues the command to the FSM
Fapi_issueDataOnly512ProgrammingCommand()	Sets up the required registers for 512-bit (32 16-bit words) programming with user provided flash data and issues the command to the FSM
Fapi_issueEccOnly64ProgrammingCommand()	Sets up the required registers for 64-bit (4 16-bit words) ECC programming with user provided ECC data and issues the command to the FSM
Fapi_issueAsyncCommand()	Issues a command (Clear Status) to FSM for operations that do not require an address

Copyright © 2024 Texas Instruments Incorporated

表 2-2. Flash 状态机（FSM）函数汇总（续）

API Function Description	Description
Fapi_checkFsmForReady()	Returns whether or not the Flash state machine (FSM) is ready or busy
Fapi_getFsmStatus()	Returns the STATCMD status register value from the Flash Wrapper

表 2-3. 读取函数汇总

API Function	Description
Fapi_doBlankCheck()	Verifies specified Flash memory range against erased state
Fapi_doVerify()	Verifies specified Flash memory range against supplied values

注

Fapi_calculatePsa() 和 Fapi_doPsaVerify() 已弃用。

表 2-4. 信息函数汇总

API Function	Description
Fapi_getLibraryInfo()	Returns the information specific to the compiled version of the API library

表 2-5. 实用函数汇总

API Function	Description
Fapi_flushPipeline()	Flushes the data cache in Flash Wrapper
Fapi_calculateEcc()	Calculates the ECC for the supplied address and 64-bit word
Fapi_isAddressEcc()	Determines if the address falls in ECC ranges
Fapi_remapEccAddress()	Remaps an ECC address to corresponding main address
Fapi_calculateFletcherChecksum()	Function calculates a Fletcher checksum for the memory range specified. Fapi_calculateFletcherChecksum() will be deprecated in future device.

2.3 API 的使用方法

本节介绍各种 API 函数的使用流程。

2.3.1 初始化流程

2.3.1.1 After Device Power Up

器件首次上电后，必须先调用 *Fapi_initializeAPI()* 函数，之后才能使用其他 API 函数（*Fapi_getLibraryInfo()* 函数除外）。此过程会根据用户指定的系统运行频率配置 Flash Wrapper。

2.3.1.2 Flash Wrapper and Bank Setup

首次执行 Flash 操作之前，必须调用 *Fapi_setActiveFlashBank()* 函数。

2.3.1.3 On System Frequency Change

首次调用 *Fapi_initializeAPI()* 函数后，如果系统运行频率发生变化，则必须再次调用此函数，然后才能使用其他 API 函数（*Fapi_getLibraryInfo()* 函数除外）。此过程会更新 API 内部状态变量。

2.3.2 使用 API 进行构建

2.3.2.1 Ojbects Library Files

Flash API 对象文件以 Arm® 标准 EABI ELF 和 COFF 对象格式提供。

Note

Compilation requires the "Enable support for GCC extensions" option to be enabled. Compiler version 6.4.0 and onwards have this option enabled by default.

Copyright © 2024 Texas Instruments Incorporated

2.3.2.2 分发文件

以下 API 文件位于 C2000Ware\libraries\flash_api\f28p55x\ 文件夹中：

- 库文件 — TMS320F28P55x Flash API 未嵌入此器件的 Boot ROM，完全由软件实现。提供的软件库采用 EABI ELF（FAPI_F28P55x_EABI_v4.00.00.lib）和 COFF（FAPI_F28P55x_COFF_v4.00.00.lib）对象格式。应用程序要擦除或编程 Flash/OTP，必须根据其使用的输出对象格式，在应用程序中包含这两个库文件之一。

- FAPI_F28P55x_EABI_v4.00.00.lib — 这是面向 TMS320F28P55x 器件、采用 EABI ELF 对象格式的 Flash API 库（构建时启用了 FPU32 标志）。
- FAPI_F28P55x_COFF_v4.00.00.lib — 这是面向 TMS320F28P55x 器件、采用 COFF 对象格式的 Flash API 库（构建时启用了 FPU32 标志）。
- 不提供定点版本的 API 库。
- 包含文件：
  – FlashTech_F28p55x_C28x.h – TMS320F28P55x 器件的主包含文件。此文件设置编译专用宏定义，然后包含 FlashTech.h 主包含文件。 hw_flash_command.h – Flash 写入/擦除保护寄存器的定义

- 以下包含文件不应由用户代码直接包含；此处列出仅供参考：

  – FlashTech.h – 列出所有公开 API 函数，并包含其他所有包含文件。 Registers.h – 各寄存器实现通用的定义，并根据所选器件类型包含相应的寄存器包含文件。 Registers_C28x.h – Flash 配置寄存器。 Types.h – 包含 API 使用的所有枚举和结构体。 Constants/F28P55x.h – F28P55x 器件的常量定义。

2.3.2.3 Key Facts For Flash API Usage

以下是一些有关 API 用法的重要信息：

- Flash API 函数名称均以“Fapi_”前缀开头。
- Flash API 不会配置 PLL。用户应用程序应根据需要配置 PLL，并将配置后的 CPUCLK 值传递给 Fapi_initialize API() 函数（本文后文会详细介绍此函数）。请注意，当系统频率小于或等于 20 MHz 时，Flash API 库不支持 Flash 擦除/编程操作。

- Flash API 不会检查 PLL 配置以确认用户输入的频率。此项工作由系统集成者负责。TI 建议使用 DCC 模块检查系统频率。实现示例请参见 C2000Ware Driverlib 时钟配置函数。

- 调用 Flash API 函数前，应根据器件数据手册配置等待状态。如果应用程序配置的等待状态与其运行频率不匹配，Flash API 会返回错误。

- Flash API 的执行可被中断。不过，在擦除/编程操作进行期间，不得从 Flash bank 读取或取指。因此，Flash API 函数、调用 Flash API 函数的用户应用程序函数，以及所有中断服务例程（ISR）都必须在 RAM 中执行。例如，除 Flash API 函数外，下方代码片段中的所有代码也必须满足上述要求。原因是 Fapi_issueAsyncCommandWithAddress() 函数会向 FSM 发出擦除命令，但不会等待擦除操作完成。只要 FSM 仍在忙于当前操作，就不得访问正在擦除的 Flash bank。

```
//  // Erase a Sector  //  oReturnCheck = Fapi_issueAsyncCommandWithAddress(Fapi_EraseSector,(uint32*)0x0080000);  //  //
Wait until the erase operation is over  //  while (Fapi_checkFsmForReady() != Fapi_Status_FsmReady){}
```


Copyright © 2024 Texas Instruments Incorporated

- Flash API 不会配置（启用/禁用）看门狗。用户应用程序可根据需要配置并喂狗。因此，不再提供 Fapi_ServiceWatchdogTimer() 函数。
- Flash API 会根据需要在内部使用 EALLOW 和 EDIS，以允许或禁止对受保护寄存器进行写入。
- 主阵列 Flash 编程必须按 64-bit 地址边界对齐（建议按 128-bit 地址边界对齐）；每个 64-bit 字在一次写入/擦除周期中只能编程一次。
- 可以分别编程数据和 ECC。不过，每个 64-bit 数据字及其对应的 ECC 字在一次写入/擦除周期中都只能编程一次。
- TMS320F28P55x 器件没有 pump semaphore。
- 不得对 link-pointer 位置编程 ECC。若编程操作的起始地址为三个 link-pointer 地址之一，API 会跳过 ECC 编程。即使用户将 Fapi_AutoEccGeneration 或 Fapi_DataAndEcc 作为编程模式参数传入，API 仍会对这些位置使用 Fapi_DataOnly 模式。Fapi_EccOnly 模式不支持对这些位置执行

  编程。用户应用程序在此处应谨慎处理。应在应用程序中为 link-pointer 位置单独设置结构体/段。不要将这些字段与其他 DCSM OTP 设置混用。如果将其他字段与 link-pointer 混在一起，API 也会跳过对非 link-pointer 位置的 ECC 编程，从而导致应用程序出现 ECC 错误。

- 使用 128-bit DCSM OTP 编程时，DCSM OTP 地址必须按 128-bit 地址边界对齐，且每个 128-bit 字只能编程一次。以下情况除外：

  – DCSM OTP 中的 DCSM Zx-LINKPOINTER1 和 Zx-LINKPOINTER2 值应

  同时编程，并可根据 DCSM 操作需要一次编程 1 bit。 ÐCSM OTP 中的 DCSM Zx-LINKPOINTER3 值可根据 DCSM 操作需要一次编程 1 bit。 如有需要，用户可单独编程 JLM_Enable。如果用户希望一次编程全部 64-bit，则必须读取 link-pointer（ZxOTP_LINKPOINTER1、ZxOTP_LINKPOINTER2、ZxOTP_LINKPOINTER3），并将该值与 JLM_Enable 值一起编程。 在给定的对齐 128-bit 字中，如果另一半对齐的 64-bit 已保留，则可以编程其中的 64-bit

---

注

在某些 512-bit 编程模式下，允许编程 DCSM OTP，但 link-pointer 地址范围除外（0x00078000 至 0x00078005 和 0x00078200 至 0x00078205）。编程 link-pointer 位置时，用户应使用 AutoECCGeneration 模式调用 Fapi_issueProgrammingCommand()。详情请参见下表。

512-Bit Programming Functions	Programing of DCSM OTP and Link Pointer Range
Fapi_issueAutoEcc512ProgrammingCommand()	DCSM OTP allowed, link pointer not allowed
Fapi_issueDataAndEcc512ProgrammingCommand()	DCSM OTP allowed, link pointer not allowed
Fapi_issueDataOnly512ProgrammingCommand()	DCSM OTP and link pointer not allowed
Fapi_issueEccOnly64ProgrammingCommand()	ECC DCSM OTP allowed, ECC link pointers not allowed

- 为避免 Zone 1 与 Zone 2 之间发生冲突，DCSM 寄存器中提供了一个信号量（FLSEM），用于配置 Flash 寄存器。用户应用程序应在初始化 Flash 并调用 Flash API 函数之前配置此信号量寄存器。有关此寄存器的详细信息，请参见 TMS320F28P55x 的 *TMS320F28P55x Real-Time Microcontrollers Technical Reference Manual*。
- 请注意，Flash API 函数不会配置任何 DCSM 寄存器。用户应用程序必须确保所需的 DCSM 设置已配置。例如，如果某个 zone 已加密保护，则必须在同一 zone 中执行 Flash API，才能擦除或编程该 zone 的 Flash 扇区；也可以先解锁该 zone。否则，Flash API 对 Flash 寄存器的写入将不会成功。Flash API 不会检查这些写入是否实际生效；它会按擦除/编程流程的要求写入寄存器，然后假定写入成功。因此，Flash API 可能返回错误的成功状态。例如，调用 Fapi_issueAsyncCommandWithAddress(Fapi_EraseSector, Address) 后可能返回成功状态，但这并不表示扇区已成功擦除。应使用 Fapi_getFSMStatus() 和 Fapi_doBlankCheck() 检查擦除状态。

- 请注意，Flash 擦除/编程操作正 在进行时，不得访问相应的 Flash bank/OTP。

Copyright © 2024 Texas Instruments Incorporated

3 API 函数

3.1 初始化函数

3.1.1 Fapi_initializeAPI()

初始化 Flash API。

函数原型

```
Fapi_StatusType Fapi_initializeAPI(                Fapi_FmcRegistersType *poFlashControlRegis
ter,          uint32 u32HclkFrequency)                      )
```

参数

poFlashControlRegister [in]	Pointer to the Flash Wrapper Registers' base address. Use FlashTech_CPU0_BASE_ADDRESS.
u32HclkFrequency [in]	System clock frequency in MHz

说明

在执行其他 Flash API 操作之前，必须先调用此函数来初始化 Flash API。系统频率或 RWAIT 发生变化时，也必须再次调用此函数。

注

RWAIT 寄存器的值必须在调用此函数之前设置。

返回值

- Fapi_Status_Success（成功）
- Fapi_Error_InvalidHclkValue（失败：系统时钟与指定的等待值不匹配）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）

示例实现

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。

3.2 Flash 状态机函数

3.2.1 Fapi_setActiveFlashBank()

初始化 Flash Wrapper，以执行擦除和编程操作。

函数原型

```
Fapi_StatusType Fapi_setActiveFlashBank(
                          Fapi_FlashBankType oNewFlashBank
                                          )
```


Copyright © 2024 Texas Instruments Incorporated

参数

oNewFlashBank [in]	Bank number to set as active. Fapi_FlashBank0 should be used for this device even though there are multiple banks.

说明

此函数会设置 Flash Wrapper，以便对 bank 执行后续操作。必须在调用 Fapi_initializeAPI() 函数之后、执行任何其他 Flash API 操作之前调用此函数。

注

应用程序只需调用一次，可使用 Fapi_FlashBank0 调用。

返回值

- Fapi_Status_Success（成功）

示例实现

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。

3.2.2 Fapi_setupBankSectorEnable()

配置扇区的写入（编程）/擦除保护。

函数原型

```
Fapi_StatusType Fapi_setupBankSectorEnable(              uint32 WEPROT_register,              uint32 oSectorMask
)
```

参数

pu32StartAddress [in]	Register address for Write/Erase protection configuration. Use *FLASH_WRAPPER_PROGRAM_BASE+FLASH_O_CMDWEPROTA* for the first 32 (0-31) sectors. Use *FLASH_WRAPPER_PROGRAM_BASE+FLASH_O_CMDWEPROTB* for the remaining main-array (32-127) sectors. Use *FLASH_WRAPPER_PROGRAM_BASE+FLASH_O_CMDWEPROT_UO* for the USER OTP.
OSectorMask [in]	32-bit mask indicating which sectors to mask from the erase and program operations.

说明

此器件中，默认情况下，所有 Flash 主阵列扇区和 USER OTP 均受到擦除和编程保护。用户应用程序必须关闭要执行擦除和/或编程操作的扇区保护。可使用此函数启用或禁用保护。每次发出擦除或编程命令之前，都应调用此函数，如 C2000Ware 中提供的 Flash API 用法示例所示。

Copyright © 2024 Texas Instruments Incorporated

此函数的第一个输入参数可以是以下三个寄存器中任意一个的地址：*CMDWEPROTA, CMDWEPROTB, CMDWEPROT_UO*

*CMDWEPROTA*该寄存器用于配置前 32 个扇区（0 至 31）的保护。此寄存器中的每一位对应一个扇区。例如：此寄存器的 Bit 0 用于配置 Sector 0 的保护，Bit 31 用于配置 Sector 31 的保护。用户提供的 32-bit 扇区掩码（传递给此函数的第二个参数）用于指示哪些扇区应从擦除和编程操作中屏蔽，即这些扇区不会被擦除或编程。如果掩码中的某一位为 1，则对应扇区不会被擦除/编程。

*CMDWEPROTB*该寄存器用于配置主阵列 Flash bank 中 32 – 127 扇区的保护。不过请注意，此寄存器中的每一位控制 8 个扇区的保护。例如，Bit 0 一起配置 32 至 39 扇区的保护，Bit 1 一起配置 40 至 47 扇区的保护，依此类推。用户提供的 32-bit 扇区掩码（传递给此函数的第二个参数）用于指示哪些扇区应从擦除和编程操作中屏蔽，即这些扇区不会被擦除或编程。如果掩码中的某一位为 1，则对应的一组扇区不会被擦除/编程；如果为 0，则会被擦除/编程。

*CMDWEPROT_UO*该寄存器用于配置 USER OTP 的保护。此寄存器中的 Bit 0 用于配置 USER OTP 保护；如果 Bit 0 配置为 1，则不会擦除/编程 USER OTP。其他位可配置为 1。由于 USER OTP 不可擦除，CMDWEPROT_UO 寄存器保护不适用于擦除操作。仅在需要时为编程操作配置此保护。

返回值

- Fapi_Status_Success（成功）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）
- Fapi_Error_FeatureNotAvailable（失败：用户请求写入未受 *CMDWEPROT* 保护的寄存器。）

示例实现（有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。另一个示例位于：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。）

3.2.3 Fapi_issueAsyncCommandWithAddress()

将用户提供的扇区地址与擦除命令一起发送给 Flash 状态机。

函数原型

```
Fapi_StatusType Fapi_issueAsyncCommandWithAddress(    Fapi_FlashStateCommandsType oCommand,
uint32 *pu32StartAddress                              )
```

参数

oCommand [in]	Command to issue to the FSM. Use Fapi_EraseSector
pu32StartAddress [in]	Flash sector address for erase operation

Copyright © 2024 Texas Instruments Incorporated

说明

此函数会针对用户提供的扇区地址，向 Flash 状态机发出擦除命令。此函数不会等待擦除操作完成；它只发送命令，然后立即返回。因此，使用 Fapi_EraseSector 命令时，此函数始终返回成功状态。用户应用程序必须等待 Flash Wrapper 完成擦除操作，然后才能再次访问 Flash。可使用 Fapi_checkFsmForReady() 函数监视已发出命令的状态。

注

此函数发出擦除命令后不会检查 STATCMD。FSM 完成擦除操作后，用户应用程序必须检查 STATCMD 值。STATCMD 用于指示擦除操作期间是否发生故障。用户应用程序可使用 Fapi_getFSMStatus 函数获取 STATCMD 值。

此外，用户应用程序应使用 Fapi_doBlankCheck() 函数确认 Flash 已擦除。

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_FeatureNotAvailable（失败：用户请求了不受支持的命令。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有关有效地址范围，请参见 *TMS320F28P55x Real-Time Microcontrollers Data Manual*TMS320F28P55x 的相关资料。

示例实现（有关 TMS320F28p55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

另一个 Flash 编程示例位于 C2000Ware：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。）

3.2.4 Fapi_issueBankEraseCommand()

将用户提供的扇区掩码与 bank 擦除命令一起发送给 Flash 状态机。

函数原型

```
Fapi_StatusType Fapi_issueBankEraseCommand(          uint32 *
pu32StartAddress                                    )
```

参数

pu32StartAddress [in]	Flash bank address for bank erase operation

说明

此函数会针对用户提供的 bank 地址，向 Flash 状态机发出 bank 擦除命令。如果 FSM 正在执行其他操作，函数会返回 FSM 忙状态；否则将继续执行 bank 擦除操作。

Copyright © 2024 Texas Instruments Incorporated

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_FlashRegsNotWritable（Flash 寄存器不可写）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见 TMS320F28P55x 的 *TMS320F28P55x Real-Time Microcontrollers Data Manual*。

示例实现（有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

另一个示例位于 C2000Ware：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。）

3.2.5 Fapi_issueProgrammingCommand()

为有效的 Flash 或 OTP 存储器地址准备数据并发出编程命令。

函数原型

```
Fapi_StatusType Fapi_issueProgrammingCommand(                    uint32 *pu32StartA
ddress,                    uint16 *pu16DataBuffer,                    uint16  u16DataB
ufferSizeInWords,                uint16 *pu16EccBuffer,                uint16  u16EccBuff
erSizeInBytes,                Fapi_FlashProgrammingCommandType oMode
)
```

参数

pu32StartAddress [in]	Start address in Flash for the data and ECC to be programmed. Also, the start address should always be even.
pu16DataBuffer [in]	Pointer to the Data buffer address. Data buffer should be 128-bit aligned.
u16DataBufferSizeInWords [in]	Number of 16-bit words in the Data buffer
pu16EccBuffer [in]	Pointer to the ECC buffer address
u16EccBufferSizeInBytes [in]	Number of 8-bit bytes in the ECC buffer
oMode [in]	Indicates the programming mode to use: Fapi_DataOnly Programs only the data buffer Fapi_AutoEccGeneration Programs the data buffer and auto generates and programs the ECC. Fapi_DataAndEcc Programs both the data and ECC buffers Fapi_EccOnly Programs only the ECC buffer

注

pu16EccBuffer 应包含与主阵列/OTP 中按 128-bit 边界对齐的地址对应的 ECC。pu16EccBuffer 的 LSB 对应主阵列较低的 64 bits，MSB 对应主阵列较高的 64 bits。

说明

此函数根据提供的参数设置 Flash 状态机的编程寄存器。
函数提供 4 种编程模式，供用户根据不同场景使用，具体用途见“不同编程模式的用途”。

Copyright © 2024 Texas Instruments Incorporated

此函数允许的编程范围见表 3-1。

表 3-1. 不同编程模式的用途

Programming Mode (oMode)	Arguments Used	Usage Purpose
Fapi_DataOnly	pu32StartAddress, pu16DataBuffer, u16DataBufferSizeInWords	Used when any custom programming utility or an user application (that embed/use Flash API) has to program data and corresponding ECC separately. Data is programmed using Fapi_DataOnly mode and then the ECC is programmed using Fapi_EccOnly mode. Generally most of the programming utilities do not calculate ECC separately and instead use Fapi_AutoEccGeneration mode. However, some Safety applications may require to insert intentional ECC errors in their Flash image (which is not possible when Fapi_AutoEccGeneration mode is used) to check the health of the SECDED (Single Error Correction and Double Error Detection) module at run time. In such case, ECC is calculated separately (using the Fapi_calculateEcc() function as applicable). Application may want to insert errors in either main array data or in the ECC as needed. In such scenarios, after the error insertion, Fapi_DataOnly mode and Fapi_EccOnly modes can be used to program the data and ECC respectively.
Fapi_AutoEccGeneration	pu32StartAddress, pu16DataBuffer, u16DataBufferSizeInWords	Used when any custom programming utility or user application (that embed/use Flash API to program Flash at run time to store data or to do a firmware update) has to program data and ECC together without inserting any intentional errors. This is the most prominently used mode.
Fapi_DataAndEcc	pu32StartAddress, pu16DataBuffer, u16DataBufferSizeInWords, pu16EccBuffer, u16EccBufferSizeInBytes	Purpose of this mode is not different than that of using Fapi_DataOnly and Fapi_EccOnly modes together. However, this mode is beneficial when both the data and the calculated ECC can be programmed at the same time.
Fapi_EccOnly	pu16EccBuffer u16EccBufferSizeInBytes	See the usage purpose given for Fapi_DataOnly mode.

表 3-2. Fapi_issueProgrammingCommand() 的允许编程范围

Flash API	Main Array	DCSM OTP	ECC	Link Pointer
Fapi_issueProgrammingCommand() 128-bit, Fapi_AutoEccGeneration mode	Allowed	Allowed	Allowed	Allowed (Flash API will program it as *Fapi_DataOnly)*
Fapi_issueProgrammingCommand() 128-bit, Fapi_DataOnly mode	Allowed	Allowed (Flash API will program it as *Fapi_AutoEccGeneration)*	Not allowed	Allowed
Fapi_issueProgrammingCommand() 128-bit, Fapi_DataAndEcc mode	Allowed	Allowed	Allowed	Allowed (Flash API will program it as *Fapi_DataOnly)*
Fapi_issueProgrammingCommand() 128-bit, Fapi_EccOnly mode	Not allowed	Not allowed	Allowed	Not allowed

注：由于上电时会启用 ECC 检查，用户必须始终为 Flash 映像编程 ECC。

Copyright © 2024 Texas Instruments Incorporated

编程模式：
Fapi_DataOnly — 此模式仅对指定地址处 Flash 中的数据部分编程。可编程范围从 1-bit 到 8 个 16-bit 字。有关 Flash 数据编程大小的限制，请参阅此函数的相关限制。待编程的起始地址加上数据缓冲区长度不得跨越 128-bit 对齐地址边界。使用此模式时，会忽略参数 4 和参数 5。

Fapi_AutoEccGeneration — 此模式会将提供的数据及自动生成的 ECC 一起写入 Flash。每个按 64-bit 存储器边界对齐的 64-bit 数据都会计算 ECC。因此，使用此模式时，给定 64-bit 对齐地址上的全部 64 bits 数据都必须同时编程。未提供的数据按全 1 处理（0xFFFF）。为 64-bit 数据计算并写入 ECC 后，即使只是将该 64-bit 数据中的某一位从 1 改为 0，也不能再次编程这些 64 bits（除非先擦除扇区），因为新的 ECC 值会与之前已编程的 ECC 值冲突。使用此模式时，如果起始地址按 128-bit 对齐，则可根据需要同时编程 8 个或 4 个 16-bit 字。如果起始地址按 64-bit 对齐但未按 128-bit 对齐，则每次只能同时编程 4 个 16-bit 字。Fapi_DataOnly 模式中的数据限制同样适用于此模式。参数 4 和参数 5 会被忽略。

注

Fapi_AutoEccGeneration 模式会将提供的数据部分与自动生成的 ECC 一起写入 Flash。ECC 根据 64-bit 对齐地址及其对应的 64-bit 数据计算。未提供的数据按 0xFFFF 处理。使用自定义编程工具逐个将代码项目输出文件中的各个段写入 Flash 时，需要考虑这一做法可能带来的实际影响。如果一个 64-bit 字跨越多个段（即包含一个段的末尾和另一个段的开头），则在编程第一个段时，不能假定 64-bit 字中缺失的数据为 0xFFFF。编程第二个段时，由于第一个 64-bit 字的 ECC 已经按缺失值为 0xFFFF 错误地计算并写入，因此将无法再为该字编程 ECC。避免此问题的一种方法是在代码项目的链接器命令文件中，将所有链接到 Flash 的段按 64-bit 边界对齐。

例如：

```
SECTIONS
   {
     .text        : > FLASH, ALIGN(4)
     .cinit       : > FLASH, ALIGN(4)
     .const       : > FLASH, ALIGN(4)
     .init_array  : > FLASH, ALIGN(4)
     .switch      : > FLASH, ALIGN(4)
   }
```

如果 Flash 中的各段未对齐，则需要跟踪每个段中的不完整 64-bit 字，并将其与其他段中的字合并，以补全该 64-bit 字。这很难实现，因此建议将各段按 64-bit 边界对齐。

某些第三方 Flash 编程工具、TI Flash 编程内核示例（*C2000Ware*）或自定义 Flash 编程方案可能会假设输入数据流均按 128-bit 对齐，并且可能不会考虑段从未对齐地址开始的情况。因此，这些工具或方案可能会假定所提供的地址已按 128-bit 对齐，并尝试一次编程尽可能多的数据（128-bits）。如果地址未对齐，就可能导致编程失败。因此建议将所有映射到 Flash 的段按 128-bit 边界对齐。

Fapi_DataAndEcc — 此模式会在指定地址处将提供的数据和 ECC 一起写入 Flash。提供的数据必须按 64-bit 存储器边界对齐，数据长度必须与提供的 ECC 相符。例如，如果数据缓冲区长度为 4 个 16-bit 字，ECC 缓冲区必须为 1 byte；如果数据缓冲区长度为 8 个 16-bit 字，ECC 缓冲区长度必须为 2 bytes。如果起始地址按 128-bit 对齐，则可根据需要同时编程 8 个或 4 个 16-bit 字。如果起始地址按 64-bit 对齐但未按 128-bit 对齐，则每次只能同时编程 4 个 16-bit 字。

pu16EccBuffer 的 LSB 对应主阵列较低的 64-bits，MSB 对应主阵列较高的 64-bits。

Copyright © 2024 Texas Instruments Incorporated

可使用 Fapi_calculateEcc() 函数，根据给定的 64-bit 对齐地址和对应数据计算 ECC。

Fapi_EccOnly — 此模式仅在指定地址处的 Flash ECC 存储空间编程 ECC 部分（此函数应提供 Flash 主阵列地址，而不是相应的 ECC 地址）。可以编程 2 bytes（ECC 存储空间位置上的 LSB 和 MSB）或 1 byte（ECC 存储空间位置上的 LSB）。

pu16EccBuffer 的 LSB 对应主阵列较低的 64-bits，MSB 对应主阵列较高的 64-bits。

使用此模式时，参数 2 和参数 3 会被忽略。

注

pu16DataBuffer 和 pu16EccBuffer 的长度分别不得超过 8 和 2。

注

此函数发出编程命令后不会检查 STATCMD。FSM 完成编程操作后，用户应用程序必须检查 STATCMD 值。STATCMD 用于指示编程操作期间是否发生故障。用户应用程序可使用 Fapi_getFsmStatus 函数获取 STATCMD 值。

此外，用户应用程序应使用 Fapi_doVerify() 函数确认 Flash 已正确编程。

此函数不会等待编程操作完成，而是仅发送命令并返回。因此，用户应用程序必须等待 Flash Wrapper 完成编程操作，然后才能再次访问 Flash。应使用 Fapi_checkFsmForReady() 函数监视已发出命令的状态。

限制

- 如上所述，只要所提供的地址按 128-bit 对齐，此函数每次最多可编程 128-bits。如果用户要编程的数据超过此数量，应循环调用此函数，每次编程 128-bits（或根据应用需要编程 64-bits）。
- 主阵列 Flash 编程必须按 64-bit 地址边界对齐；每个 64-bit 字在一次写入或擦除周期中只能编程一次。
- 可以分别编程数据和 ECC。不过，每个 64-bit 数据字及其对应的 ECC 字在一次写入或擦除周期中都只能编程一次。
- 不得对 link-pointer 位置编程 ECC。无论用户选择 Fapi_AutoEccGeneration 还是 Fapi_DataAndEcc 模式，API 都会对这些位置发出 Fapi_DataOnly 命令。Fapi_EccOnly 模式不支持 link-pointer 位置。
- Bank0 DCSM OTP 空间不得使用 Fapi_EccOnly 模式，否则会返回错误。对于 DCSM OTP 空间，应使用 Fapi_AutoEccGeneration 或 Fapi_DataAndEcc 编程模式
  。

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_AsyncIncorrectDataBufferLength（失败：指定的数据缓冲区大小不正确。如果在编程 Bank0 DCSM OTP 空间时选择 Fapi_EccOnly 模式，也会返回此错误。）
- Fapi_Error_AsyncIncorrectEccBufferLength（失败：指定的 ECC 缓冲区大小不正确。）
- Fapi_Error_AsyncDataEccBufferLengthMismatch（失败：数据缓冲区大小未按 64-bit 对齐，或数据长度跨越了 128-bit 对齐的存储器边界。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）

Copyright © 2024 Texas Instruments Incorporated

- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的模式。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见 *TMS320F28P55x Real-Time Microcontrollers Data Manual*

  TMS320F28P55x 的相关资料。

示例实现

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

有关 TMS320F28P55x 的 Flash 编程示例，请参见 C2000Ware 中提供的示例：“C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。

3.2.6 Fapi_issueProgrammingCommandForEccAddresses()

将 ECC 地址重新映射为数据地址，并调用 Fapi_issueProgrammingCommand()。

函数原型

```
Fapi_StatusType  Fapi_issueProgrammingCommandForEccAddresses(
        uint32 *pu32StartAddress,
        uint16 *pu16EccBuffer,
        uint16   u16EccBufferSizeInBytes
)
```

参数

pu32StartAddress [in]	ECC start address in Flash for the ECC to be programmed
pu16EccBuffer [in]	Pointer to the ECC buffer address
u16EccBufferSizeInBytes [in]	Number of bytes in the ECC buffer If the number of bytes is 1, LSB (ECC for lower 64 bits) gets programmed. MSB alone cannot be programmed using this function. If the number of bytes is 2, both LSB and MSB bytes of ECC get programmed.

说明

此函数会将 ECC 存储器空间中的地址重新映射为对应的数据地址空间地址，然后调用 Fapi_issueProgrammingCommand() 写入提供的 ECC 数据。对于使用 Fapi_EccOnly 模式调用 Fapi_issueProgrammingCommand() 时适用的限制，同样适用于此函数。pu16EccBuffer 的 LSB 对应主阵列的低 64 bits，MSB 对应

主阵列的高 64 bits。

Note

The length of the pu16EccBuffer cannot exceed 2.

注意

此函数发出编程命令后不会检查 STATCMD。FSM 完成编程操作后，用户应用必须检查 STATCMD 的值，以确定编程过程中是否发生故障。用户应用可调用 Fapi_getFSMStatus 函数获取 STATCMD 的值。

注意

Bank0 DCSM OTP 空间不得使用 Fapi_EccOnly 模式，否则会返回错误。对于 DCSM OTP 空间，应使用 Fapi_AutoEccGeneration 或 Fapi_DataAndEcc 编程模式。

Copyright © 2024 Texas Instruments Incorporated

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_AsyncIncorrectEccBufferLength（失败：指定的数据缓冲区大小不正确）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）

Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Real-Tim Microcontrollers Data Manual*。）

3.2.7 Fapi_issueAutoEcc512ProgrammingCommand()

设置数据，并发出 512-bit（32 个 16-bit 字）的 AutoEcc generation mode 编程命令，以便对有效的 Flash 或 OTP 存储器地址进行编程。

函数原型

```
Fapi_StatusType Fapi_issueAutoEcc512ProgrammingCommand (          uint32 *pu32StartAddress,uint
16*pu16DataBuffer,          uint16u16DataBufferSizeInWords          )
```

参数

pu32StartAddress [in]	Start address in Flash for the data and ECC to be programmed.
pu16DataBuffer [in]	Pointer to the Data buffer address. Address of the Data buffer should be 512-bit aligned.
u16DataBufferSizeInWords [in]	Number of 16-bit words in the Data buffer. Max Databuffer size in words should not exceed 32.

说明

此函数会根据用户提供的 512-bit 数据（第二个参数）自动生成 8 bytes 的 ECC 数据，并将数据和 ECC 一并编程到用户指定的、按 512-bit 对齐的 Flash 地址（第一个参数）。发出此命令后，Flash 状态机将对全部 512 bits 及 ECC 进行编程。因此，使用此模式时，未提供的数据将按全 1（0xFFFF）处理。对于某个 512-bit 数据块，一旦其 ECC 计算并写入，除非擦除所在扇区，否则不得再次对这 512 bits 编程；即使只是将其中的某一位从 1 编程为 0 也不行，因为新的 ECC 值会与此前已写入的 ECC 值冲突。

注意

Fapi_issueAutoEcc512ProgrammingCommand() 函数会将提供的数据部分与自动生成的 ECC 一并写入 Flash。ECC 根据按 512-bit 对齐的地址及其对应的 512-bit 数据计算。未提供的数据均按 0xFFFF 处理。使用自定义编程工具逐段读取代码项目的输出文件并写入 Flash 时，需要特别注意这一点：如果一个 512-bit 字跨越多个 section（即包含一个 section 的末尾和另一个 section 的开头），则在写入第一个 section 时，不能假定 64-bit 字中的缺失数据为 0xFFFF。等到写入第二个 section 时，第一个 512-bit 字的 ECC 已按缺失数据为 0xFFFF 的假设错误计算并写入，因此无法再为其编程 ECC。避免此问题的一种方法是在代码项目的 linker command file 中，将所有链接到 Flash 的 section 按 512-bit 边界对齐。

SECTIONS（段）

```
 {
    .text      : > FLASH, ALIGN(32)
    .cinit     : > FLASH, ALIGN(32)
    .const     : > FLASH, ALIGN(32)
```


Copyright © 2024 Texas Instruments Incorporated

```
.init_array : > FLASH, ALIGN(32)
.switch     : > FLASH, ALIGN(32)
}
```

如果不对 Flash 中的 section 进行对齐，就必须跟踪每个 section 中未完整的 512-bit 字，并将其与其他 section 中用于补齐该 512-bit 字的数据合并；这很难实现。因此，建议将各 section 按 512-bit 边界对齐。

某些第三方 Flash 编程工具、TI Flash 编程内核示例（***C2000Ware***）或自定义 Flash 编程方案可能会假设输入数据流始终按 512-bit 对齐，因此不会预料到某个 section 可能从未对齐的地址开始。此类工具可能会假设所提供的地址已按 512-bit 对齐，并尝试一次编程尽可能多的 512-bit 字。当地址未对齐时，这可能导致编程失败。因此，建议将所有映射到 Flash 的 section 按 512-bit 边界对齐。

有关此函数允许的编程范围，请参见表 3-3。

表 3-3. Fapi_issueAutoEcc512ProgrammingCommand() 的允许编程范围

FlashAPI	Main Array	DCSM OTP	ECC	Link pointer
Fapi_issueAutoEcc512ProgrammingCommand()	Allowed	Allowed	Allowed	Not allowed

限制

- 如上所述，此函数每次最多只能编程 512-bits（前提是所提供的地址按 512-bit 边界对齐）。如果用户需要编程更多数据，应循环调用此函数，每次编程 512-bits。
- 主阵列 Flash 的编程地址必须按 512-bit 边界对齐；每个 32 个 16-bit 字组成的块在一次写入或擦除周期中只能编程一次。
- 凡是从 link pointer 地址开始的 512-bit 地址范围，都必须始终通过 128-bit Fapi_issueProgrammingCommand() 进行编程。

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_AsyncIncorrectDataBufferLength（失败：指定的数据缓冲区大小不正确。如果在编程 Bank0 DCSM OTP 空间时选择 Fapi_EccOnly 模式，也会返回此错误。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）
- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的模式。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Microcontrollers Data Manual*。）

示例实现

此函数的示例实现将在下一个 C2000ware 版本中发布。

Copyright © 2024 Texas Instruments Incorporated

3.2.8 Fapi_issueDataAndEcc512ProgrammingCommand()

配置 Flash 状态机寄存器，以使用用户提供的 Flash 数据和 ECC 数据执行 512-bit（32 个 16-bit 字）编程，并向有效的 Flash 和 OTP 存储器发出编程命令。

函数原型

```
Fapi_StatusType Fapi_issueDataAndEcc512ProgrammingCommand(          uint32
*pu32StartAddress,                  uint16 *pu16DataBuffer,         uint16 u16
DataBufferSizeInWords,              uint16 *pu16EccBuffer,          uint16
u16EccBufferSizeInBytes                         )
```

参数

pu32StartAddress [in]	用于写入数据和 ECC 的 Flash 地址，必须按 512-bit 边界对齐。
pu16DataBuffer [in]	指向数据缓冲区的指针。数据缓冲区地址必须按 512-bit 边界对齐。
u16DataBufferSizeInWords [in]	数据缓冲区中的 16-bit 字数，最多不得超过 32 个字。
pu16EccBuffer [in]	指向 ECC 缓冲区的指针。
u16EccBufferSizeInBytes [in]	ECC 缓冲区中的 8-bit 字节数；以字为单位的缓冲区容量不得超过 8。

说明

此函数会将用户提供的 512-bit 数据（第二个参数）和 8 字节 ECC 数据（第四个参数）写入用户指定的 Flash 地址。该地址必须按 512-bit 边界对齐；数据长度也必须与提供的 ECC 长度相匹配。换言之，如果数据缓冲区长度为 32 个 16-bit 字，ECC 缓冲区必须为 8 字节（每个 64-bit 数据对应 1 字节 ECC）。

pu16EccBuffer 中的每个字节都对应 pu16DataBuffer 所提供主阵列数据中的一个 64-bit 数据单元。

详情请参见表 3-6。

可使用 Fapi_calculateEcc() 函数，根据按 64-bit 对齐的地址及其对应数据计算 ECC。

有关此函数允许的编程范围，请参见表 3-4。

表 3-4. Fapi_issueDataAndEcc512ProgrammingCommand() 的允许编程范围

Flash API	主阵列	DCSM OTP	ECC	链接指针
Fapi_issueDataAndEcc512 ProgrammingCommand()	允许	允许	允许	不允许

限制
- 如上所述，此函数每次最多编程 512 位，且所提供地址必须按 512-bit 边界对齐。如需编程更多数据，请循环调用此函数，每次最多编程 512 位。
- 主阵列 Flash 的编程地址必须按 512-bit 边界对齐；在一次写入/擦除周期内，每个由 32 个 16-bit 字组成的数据块只能编程一次。
- 从 link pointer 地址开始的 512-bit 地址范围，必须始终通过 128-bit Fapi_issueProgrammingCommand() 编程。

- 主阵列 Flash 编程地址必须按 512-bit 边界对齐；每个包含 32 个 16-bit 字的数据块在一次写入或擦除周期中只能编程一次。
- 凡是从 link pointer 地址开始的 512-bit 地址范围，都必须始终通过 128-bit Fapi_issueProgrammingCommand() 进行编程。

Copyright © 2024 Texas Instruments Incorporated

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_AsyncIncorrectDataBufferLength（失败：指定的数据缓冲区大小不正确。如果在编程 Bank0 DCSM OTP 空间时选择 Fapi_EccOnly 模式，也会返回此错误。）
- Fapi_Error_AsyncIncorrectEccBufferLength（失败：指定的 ECC 缓冲区大小不正确。）
- Fapi_Error_AsyncDataEccBufferLengthMismatch（失败：数据缓冲区大小未按 64-bit 对齐，或数据长度跨越了 128-bit 对齐的存储器边界。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）
- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的模式。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Microcontrollers Data Manual*。）

示例实现

示例实现将在下一个 C2000ware 版本中发布。

3.2.9 Fapi_issueDataOnly512ProgrammingCommand()

配置 Flash 状态机寄存器，以使用用户提供的 512-bit（32 个 16-bit 字）数据进行编程，并向有效 Flash 存储器发出编程命令。

函数原型

```
Fapi_StatusType Fapi_issueDataOnly512ProgrammingCommand(
uint32 *pu32StartAddress,                uint16 *pu16DataBuffer,
uint16 u16DataBufferSizeInWords                                  )
```

参数

pu32StartAddress [in]	512-bit aligned flash address to program the provided data
pu16DataBuffer [in]	Pointer to the Data buffer address. Data buffer should be 512-bit aligned.
u16DataBufferSizeInWords [in]	Number of 16-bit words in the Data buffer. Max Data buffer size in words should not exceed 32.

说明

此函数只会将数据部分编程到指定的 Flash 地址。它可以将 512-bit 数据（第二个参数）编程到用户提供的、按 512-bit 对齐的 Flash 地址。

当嵌入或使用 Flash API 的用户应用需要分别编程 512-bit 数据和对应的 64-bit ECC 数据时，可使用此函数。先调用 Fapi_issueDataOnly512ProgrammingCommand() 函数编程 512-bit 数据，再调用 Fapi_issueEccOnly64ProgrammingCommand() 函数编程 64-bit ECC。通常，大多数编程工具不会单独计算 ECC，而是使用 Fapi_issueAutoEcc512ProgrammingCommand() 函数。不过，某些安全应用可能需要在其 Flash 映像中故意插入 ECC 错误（使用 Fapi_AutoEccGeneration 模式时无法实现这一点），以便在运行时检查 SECDED（单错误纠正和双错误检测）模块的健康状态。在这种情况下，应单独计算 ECC（可使用 Fapi_calculateEcc() 函数）。应用可根据需要在主阵列数据或 ECC 中插入错误。此类情况下，插入错误后可调用 Fapi_issueDataOnly512ProgrammingCommand() API 编程数据，再调用 Fapi_issueEccOnly64ProgrammingCommand() API 编程 64-bit ECC。

Copyright © 2024 Texas Instruments Incorporated

有关此函数允许的编程范围，请参见表 3-5。

表 3-5. Fapi_issueDataOnly512ProgrammingCommand() 的允许编程范围

Flash API	Main Array	DCSM OTP	ECC	Link Pointer
Fapi_issueDataOnly512ProgrammingCommand()	Allowed	Not allowed	Not allowed	Not allowed

限制

- 如上所述，此函数每次最多只能编程 512-bits（前提是所提供的地址按 512-bit 边界对齐）。如果用户需要编程更多数据，应循环调用此函数，每次编程 512-bits。

- 主阵列 Flash 编程地址必须按 512-bit 边界对齐；每个包含 32 个 16-bit 字的数据块在一次写入或擦除周期中只能编程一次。
- 凡是从 link pointer 地址开始的 512-bit 地址范围，都必须始终通过 128-bit Fapi_issueProgrammingCommand() 进行编程。

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_AsyncIncorrectDataBufferLength（失败：指定的数据缓冲区大小不正确。如果在编程 Bank0 DCSM OTP 空间时选择 Fapi_EccOnly 模式，也会返回此错误。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）

- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的模式。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Real-Time Microcontrollers Data Manual*。）

示例实现

此函数的示例实现将在下一个 C2000ware 版本中发布。

3.2.10 Fapi_issueEccOnly64ProgrammingCommand()

配置 Flash 状态机寄存器，以使用用户提供的 ECC 数据执行 64-bit（4 个 16-bit 字）编程，并向有效的 Flash 和 OTP 存储器发出编程命令。

函数原型

```
Fapi_StatusType Fapi_issueEccOnly64ProgrammingCommand(
uint32 *pu32StartAddress,          uint16 *pu16EccBuffer,
uint16 u16EccBufferSizeInBytes                          )
```

参数

pu32StartAddress [in]	512-bit aligned flash address to program the provided ECC data
pu16EccBuffer [in]	Pointer to the ECC buffer address
u16EccBufferSizeInBytes[in]	Number of 8-bit bytes in the ECC buffer. Max Eccbuffer size in words should not exceed 8.

Copyright © 2024 Texas Instruments Incorporated

说明

此函数只会将 ECC 部分编程到指定地址在 Flash ECC 存储器空间中的位置（调用此函数时应提供 Flash 主阵列地址，而不是对应的 ECC 地址）。它可以编程
64-bit ECC 数据（第二个参数），写入到与用户提供的、按 512-bit 对齐的
Flash 地址相对应的 ECC 地址。

64-bit ECC 数据可拆分为 8 bytes ECC 数据，对应按 512-bit 对齐的数据（4 × 128，每个 2 bytes 对应一组 128 -bit 数据）。详情请参见表 3-6。

表 3-6. 64-bit ECC 数据解读

512 Bits Data (4 * 128bits)			
1st 128-Bit Data	2nd 128-Bit Data	3rd 128-Bit Data	4th 128-Bit Data
LSB of pu16EccBuffer[0]	LSB of pu16EccBuffer[1]	LSB of pu16EccBuffer[2]	LSB of pu16EccBuffer[3]
MSB of pu16EccBuffer[0]	MSB of pu16EccBuffer[1]	MSB of pu16EccBuffer[2]	MSB of pu16EccBuffer[3]

有关此函数允许的编程范围，请参见表 3-7。

表 3-7. Fapi_issueEccOnly64ProgrammingCommand() 的允许编程范围

Flash API	Main Array	DCSM OTP	ECC	Link Pointer
Fapi_issueEccOnly64ProgrammingCommand()	Not Allowed	Not Allowed	Allowed	Not Allowed

限制

- 如上所述，此函数每次最多只能编程 64-bits ECC。如果用户需要编程更多数据，应循环调用此函数，每次编程 64-bits。
- 主阵列 Flash 编程必须按 512-bit 地址边界对齐；每个 64-bit ECC 字只能 在一次写入或擦除周期中编程一次。
- 不得对 link pointer 位置编程 ECC。从 link pointer 地址开始的 512-bit 地址范围，必须始终通过 128-bit Fapi_issueProgrammingCommand() 进行编程。

返回值

- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Status_Success（成功）
- Fapi_Error_AsyncDataEccBufferLengthMismatch（失败：数据缓冲区大小未按 64-bit 对齐，或数据长度跨越了 128-bit 对齐的存储器边界。）
- Fapi_Error_FlashRegsNotWritable（失败：Flash 寄存器写入失败。用户应确保 API 在与 Flash 操作目标地址相同的 zone 中执行，或者在 Flash 操作前解锁该 zone。）
- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的模式。）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Microcontrollers Data Manual*。）

示例实现

此函数的示例实现将在下一个 C2000ware 版本中发布。

3.2.11 Fapi_issueAsyncCommand()

向 Flash 状态机发出命令。此函数可发出的命令列表， 请参见说明。

函数原型

```
Fapi_StatusType Fapi_issueAsyncCommand(        Fapi_FlashStateComm
andsType oCommand                              )
```


Copyright © 2024 Texas Instruments Incorporated

参数

oCommand [in]	Command to issue to the FSM. Use Fapi_ClearStatus command.

说明

此函数用于向 Flash 状态机发出无需额外信息（例如地址）的命令。在此器件上，应使用此函数向 Flash 状态机发出 Fapi_ClearStatus 命令。注意：每次发出编程或擦除命令前，都应发出 Fapi_ClearStatus 命令，如 C2000Ware 提供的 Flash 编程示例所示。只有 STATCMD 为 0 时才能发出新的编程或擦除命令；发出 Fapi_ClearStatus 命令即可将 STATCMD 清零。

返回值

- Fapi_Status_Success（成功）
- Fapi_Status_FsmBusy（FSM 忙）
- Fapi_Error_FeatureNotAvailable（失败：用户传入了不受支持的命令。）

示例实现

TMS320F28P55x 的 Flash 编程示例请参见 C2000Ware 中的 “C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_128bit_programming.c”。

TMS320F28P55x 的 Flash 编程示例请参见 C2000Ware 中的 “C2000Ware_.....\driverlib\f28p55x\examples\flash\flashapi_512bit_programming.c”。

3.2.12 Fapi_checkFsmForReady()

返回 Flash 状态机的状态。

函数原型

```
Fapi_StatusType Fapi_checkFsmForReady(void)
```

参数

无

说明

此函数返回 Flash 状态机的状态，用于指示状态机是否已准备好接受新命令。此函数主要用于检查擦除或编程操作是否已完成。

返回值

- Fapi_Status_FsmBusy（FSM 正忙，不能接受新命令，但可接受挂起命令）
- Fapi_Status_FsmReady（FSM 已准备好接受新命令）

3.2.13 Fapi_getFsmStatus()

返回 STATCMD 寄存器的值。

函数原型

```
Fapi_FlashStatusType Fapi_getFsmStatus(void)
```

参数

无

Copyright © 2024 Texas Instruments Incorporated

说明

此函数返回 STATCMD 寄存器的值。用户应用可通过该寄存器判断擦除或编程操作是已成功完成、正在进行、已挂起还是已失败。每次擦除或编程操作后，用户应用都应检查此寄存器的值，以确认是否发生故障。

表 3-8. STATCMD 寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			FAILMISC				FAILINVDATA		FAILILLADDR	FAILVERIFY	FAILWEPROT		CMDINPROGRESS	CMDPASS	CMDDONE
			RO[1] - 0x0				RO - 0x0		RO - 0x0	RO - 0x0	RO - 0x0		RO - 0x0	RO - 0x0	RO - 0x0

(1) RO - 只读

表 3-9. STATCMD 寄存器字段说明

Bit	Name	Description	Reset Value
12	FAILMISC	Command failed due to error other than write/erase protect violation or verify error. 0: No Fail 1: Fail	0x0
8	FAILINVDATA	Program command failed because an attempt was made to program a stored 0 value to a 1. 0: No Fail 1: Fail	0x0
6	FAILILLADDR	Command failed due to the use of an illegal address. 0: No Fail 1: Fail	0x0
5	FAILVERIFY	Command failed due to verify error. 0: No Fail 1: Fail	0x0
4	FAILWEPROT	Command failed due to Write/Erase Protect Sector violation. 0: No Fail 1: Fail	0x0
2	CMDINPROGRESS	Command in Progress 0: Complete 1: In Progress	0x0
1	CMDPASS	Command Pass - valid when CMD_DONE field is 1 0: Fail 1: Pass	0x0
0	CMDDONE	Command Done 0: Not Done 1: Done	0x0

3.3 读取函数

3.3.1 Fapi_doBlankCheck()

检查指定区域是否处于已擦除状态。

函数原型

```
Fapi_StatusType Fapi_doBlankCheck(          uint32 *pu32StartAddress,          u
int32  u32Length,          Fapi_FlashStatusWordType *poFlashStatusWord
)
```


Copyright © 2024 Texas Instruments Incorporated

参数

pu32StartAddress [in]	Start address for region to blank check
u32Length [in]	Length of region in 32-bit words to blank check
poFlashStatusWord [out]	Returns the status of the operation if result is not Fapi_Status_Success
->au32StatusWord[0]	Address of first non-blank location
->au32StatusWord[1]	Data read at first non-blank location
->au32StatusWord[2	Value of compare data (always 0xFFFFFFFF)
->au32StatusWord[3]	N/A

说明

此函数从指定地址开始，按指定的 32-bit 字数量检查相应长度的 Flash 是否为空（即处于已擦除状态）。如果发现非空位置，函数会将相应的地址和数据写入 poFlashStatusWord 参数。

限制

无

返回值

- Fapi_Status_Success（成功）- 指定的 Flash 位置处于已擦除状态
- Fapi_Error_Fail（失败：指定区域非空）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见 *TMS320F28P55x Real-Time Microcontrollers Data Manual*适用于 TMS320F28P55x 的相关资料。

3.3.2 Fapi_doVerify()

将指定区域与提供的数据进行校验。

函数原型

```
Fapi_StatusType Fapi_doVerify(        uint32 *pu32StartAddress,        uint3
2  u32Length,        uint32 *pu32CheckValueBuffer,        Fapi_FlashStatus
WordType *poFlashStatusWord                        )
```

参数

pu32StartAddress [in]	Start address for region to verify
u32Length [in]	Length of region in 32-bit words to verify
u32Length [in]	Address of buffer to verify region against. Data buffer should be 128-bit aligned.
poFlashStatusWord [out]	Returns the status of the operation if result is not Fapi_Status_Success
->au32StatusWord[0]	Address of first verify failure location
->au32StatusWord[1]	Data read at first verify failure location
->au32StatusWord[2]	Value of compare data
->au32StatusWord[3]	N/A

说明

此函数从指定地址开始，按指定的 32-bit 字数量将器件内容与提供的数据进行校验。如果某个位置校验不匹配，函数会将结果写入 poFlashStatusWord 参数。

Copyright © 2024 Texas Instruments Incorporated

限制

无

返回值

- Fapi_Status_Success（成功：指定区域与提供的数据匹配）
- Fapi_Error_Fail（失败：指定区域与提供的数据不匹配）
- Fapi_Error_InvalidAddress（失败：用户提供了无效地址。有效地址范围请参见适用于 TMS320F28P55x 的 *TMS320F28P55x Real-Time Microcontrollers Data Manual*。）

3.4 信息查询函数

3.4.1 Fapi_getLibraryInfo()

返回此次编译的 Flash API 信息。

函数原型

```
Fapi_LibraryInfoType Fapi_getLibraryInfo(void)
```

参数

无

说明

此函数返回与 Flash API 库本次编译相关的信息，数据保存在 Fapi_LibraryInfoType 结构体中。该结构体包含以下成员：

- u8ApiMajorVersion — 此次 API 编译的主版本号。此值为 2。
- u8ApiMinorVersion — 此次 API 编译的次版本号。对于 F28P55x 器件，次版本号为 0。
- u8ApiRevision — 此次 API 编译的修订版本号。

此发行版的修订号为 10。

- oApiProductionStatus — 此次编译的 API 生产状态（Alpha_Internal、Alpha、Beta_Internal、Beta、Production）。

此发行版的生产状态为 Production。

- u32ApiBuildNumber — 此次编译的构建号。
- u8ApiTechnologyType — 指示 API 所支持的 Flash 技术。此字段返回 0x5。
- u8ApiTechnologyRevision — 指示 API 所支持技术的修订版本。
- u8ApiEndianness — 对于 F28P55x 器件，此字段始终返回 1（Little Endian，小端序）。
- u32ApiCompilerVersion — 用于编译 API 的 Code Composer Studio 代码生成工具版本号。

返回值

- Fapi_LibraryInfoType（包含此次 API 编译的相关信息）

3.5 实用函数

3.5.1 Fapi_flushPipeline()

刷新 Flash Wrapper 流水线缓冲区。

Copyright © 2024 Texas Instruments Incorporated

函数原型

```
void Fapi_flushPipeline(void)
```

参数

无

说明

此函数用于刷新 Flash Wrapper 数据缓存。在擦除或编程操作后，首次通过 API 以外的方式读取 Flash 前，必须刷新数据缓存。

返回值

无

3.5.2 Fapi_calculateEcc()

根据提供的地址和 64-bit 值计算 ECC。

函数原型

```
uint8 Fapi_calculateEcc(          uint32 u32
Address,          uint64 u64Data
)
```

参数

u32Address [in]	Address of the 64-bit value to calculate the ECC
u64Data [in]	64-bit value to calculate ECC on (should be in little endian order)

说明

此函数会根据地址计算按 64-bit 对齐的数据字的 ECC。现在无需向此函数提供左移后的地址，Flash API 会负责处理。

返回值

- 计算得到的 8-bit ECC（应忽略 16-bit 返回值的高 8 bits）
- 发生错误时，16-bit 返回值为 0xDEAD。

3.5.3 Fapi_isAddressEcc()

指示某个地址是否位于 Flash Wrapper ECC 地址空间。

函数原型

```
boolean Fapi_isAddressEcc(          uint3
2 u32Address     )
```

参数

u32Address [in]	Address to determine if it lies in ECC address space

说明

如果地址位于 ECC 地址空间，此函数返回 True；否则返回 False。

Copyright © 2024 Texas Instruments Incorporated

返回值

- FALSE（地址不在 ECC 地址空间中）
- TRUE（地址位于 ECC 地址空间中）

3.5.4 Fapi_remapEccAddress()

将 ECC 地址重新映射到主阵列地址空间。

函数原型

```
uint32 Fapi_remapEccAddress(        uint32
u32EccAddress     )
```

参数

`u32EccAddress [in] ECC address to remap`

说明

此函数根据给定的 Flash ECC 地址返回对应的主阵列 Flash 地址。用户需要在已知 ECC 地址处编程 ECC 数据时，可调用此函数获取对应的主阵列地址。注意：调用 Fapi_issueProgrammingCommand() 时必须提供主阵列地址，而不是 ECC 地址（即使使用 Fapi_EccOnly 模式也是如此）。

返回值

- 32-bit 主阵列 Flash 地址。
- 如果提供的 Flash ECC 地址无效，则返回 0xFFFFFFFF。

3.5.5 Fapi_calculateFletcherChecksum()

根据给定的地址和长度计算 Fletcher 校验和。

函数原型

```
uint32 Fapi_calculateFletcherChecksum(          uint1
6 *pu16Data,          uint16 u16Length
)
```

参数

pu16Data [in]	Address to start calculating the checksum from
u16Length [in]	Number of 16-bit words to use in calculation

说明

此函数从提供的地址开始，按指定的 16-bit 字数量生成 32-bit Fletcher 校验和。注意：此函数仅适用于 Flash 主阵列地址范围，不得提供 DCSM OTP 地址范围。

返回值

- 32-bit Fletcher 校验和值

4 建议的 FSM 流程

4.1 出厂新器件

器件出厂时已擦除。建议对收到的器件执行空白检查，以确认其已擦除，但这并非必需。

Copyright © 2024 Texas Instruments Incorporated

4.2 建议的擦除流程

图 4-1 展示了擦除器件中一个或多个扇区的流程。更多信息请参见第 3.2.11、3.2.2 和 3.2.3 节。

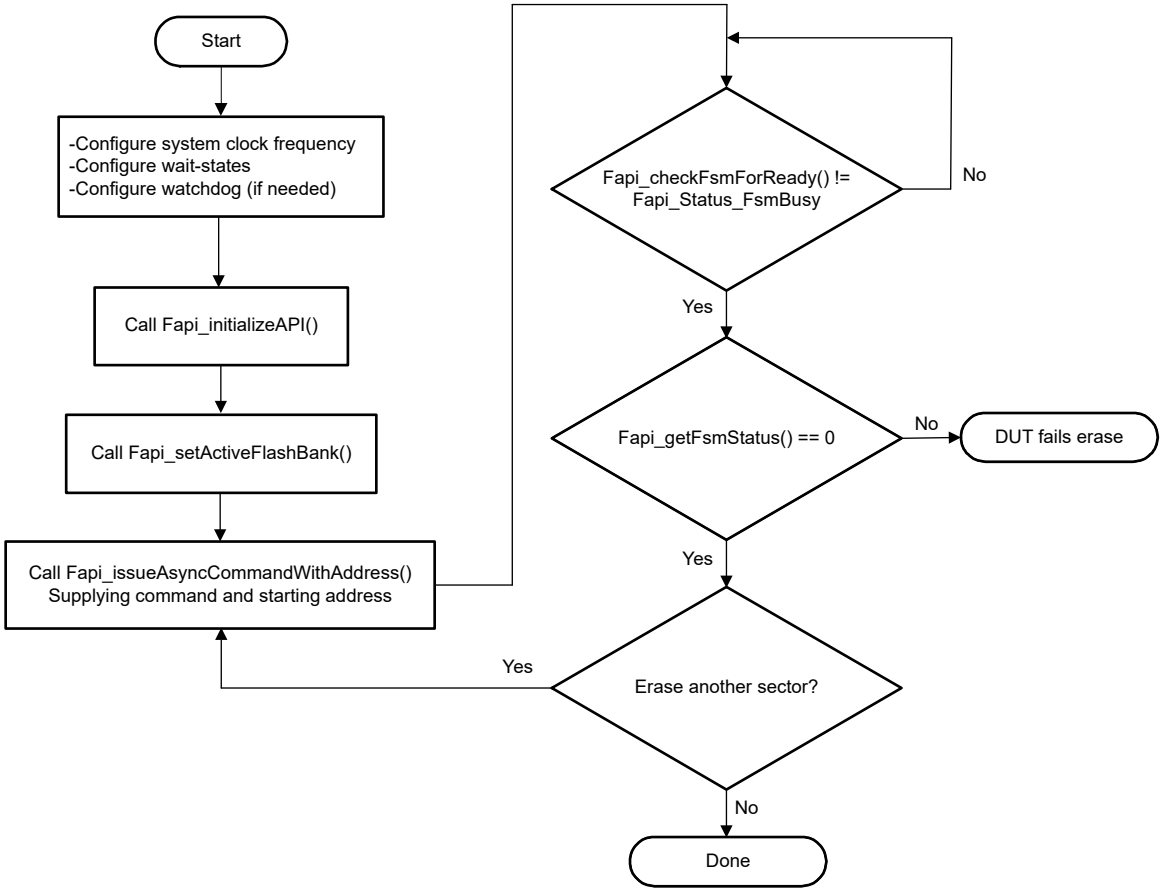


图 4-1. 建议的擦除流程

注意：发出扇区擦除命令前，应先发出 Fapi_ClearStatus 命令，并根据应用需要配置相应扇区的写入/擦除保护。

4.3 建议的 Bank 擦除流程

图 4-2 展示了擦除一个 Flash bank 的流程。更多信息请参见第 3.2.11、3.2.2 和 3.2.3 节。

Copyright © 2024 Texas Instruments Incorporated

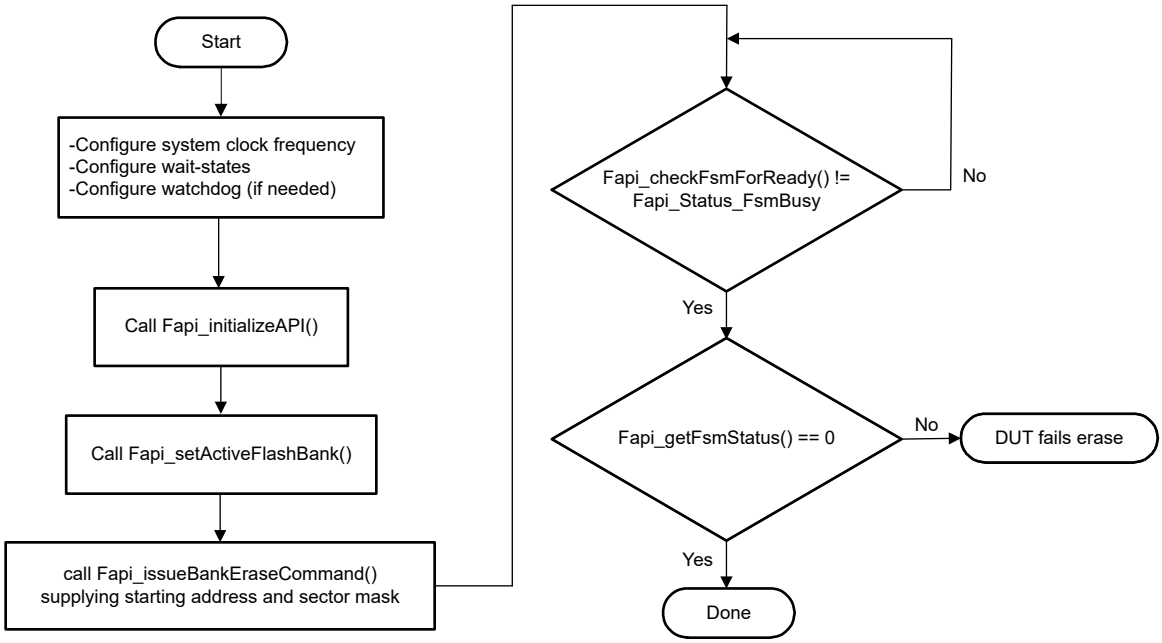


图 4-2. 建议的 Bank 擦除编程流程

Note

Before issuing the program command, Fapi_ClearStatus command should be issued and the Write/Erase protections should be configured for the sectors as needed by the application.

Copyright © 2024 Texas Instruments Incorporated

4.4 建议的编程流程

图 4-3 展示了器件编程流程。该流程假定用户已按照建议的擦除流程擦除了所有受影响的扇区或 bank。更多信息请参见第 3.2.11、3.2.2 和 3.2.3 节。

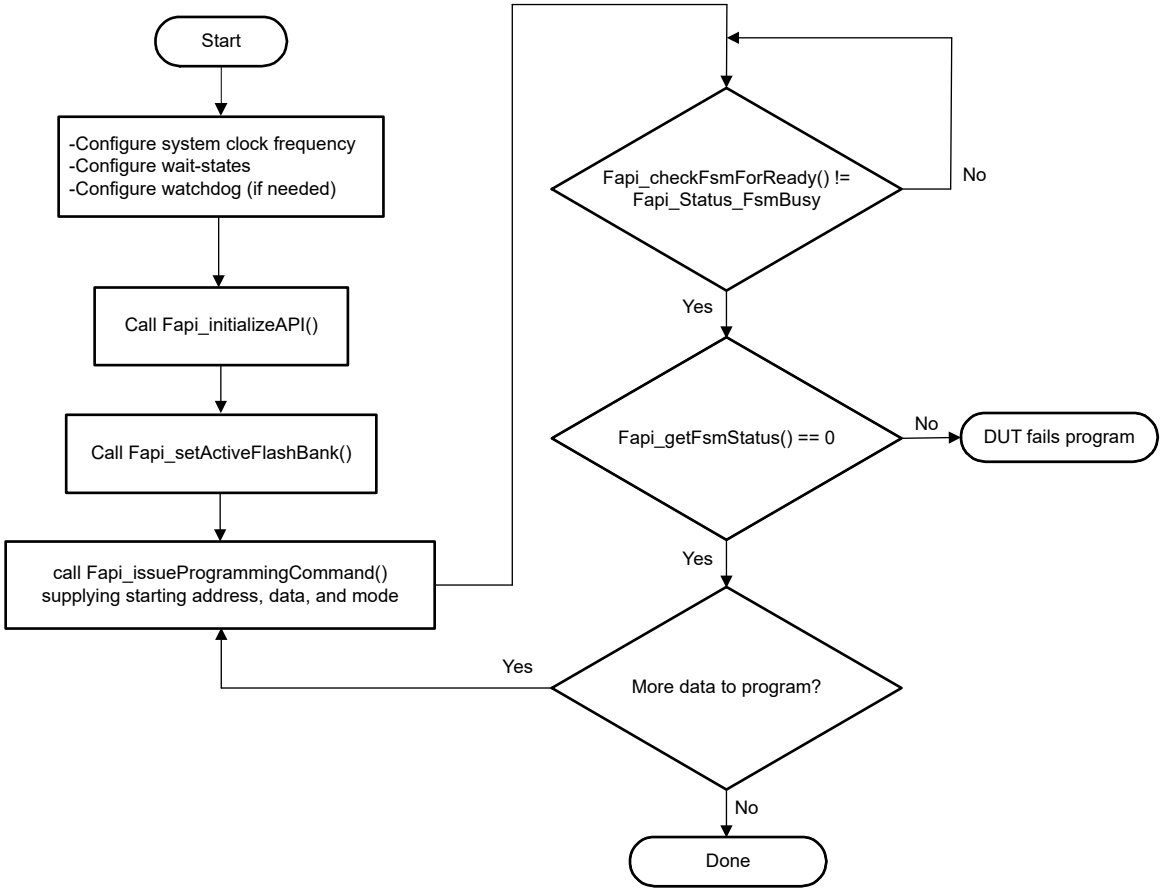


图 4-3. 建议的编程流程

Note

Before issuing the program command, Fapi_ClearStatus command should be issued and the Write/Erase protections should be configured for the sectors as needed by the application.

Copyright © 2024 Texas Instruments Incorporated

5 与安全相关的软件应用使用假设

1. 使用其他 Flash API 函数前，必须先调用一次 Fapi_initializeAPI()。无需重复调用此函数。但如果用户应用在运行时更改系统频率或 Flash 等待状态配置（这种情况较少见），则必须再次调用此函数。2. 调用 Fapi_initializeAPI() 前，用户应用必须确保 PLL 已正确配置为所需的输出频率。DriverLib 提供的 SysCtl_setClock() 会使用 DCC 模块完成此配置。SysCtl_setClock() 调用的 SysCtl_isPLLValid() 会使用 DCC 模块确认 PLL 输出处于预期范围内。有关 DCC 模块的详细信息，请参见器件专用的技术参考手册。3. 如果用户应用需要保护 Flash 寄存器空间，避免因代码失控等原因而损坏，可使用 ERAD 模块实现。有关 ERAD 的使用说明，请参见器件专用的技术参考手册。4. 对 Flash 主阵列地址范围使用 Fapi_BlankCheck() 时，建议同时检查对应的 ECC 阵列地址范围。每个扇区的 ECC 地址范围列在器件专用数据手册的 Memory Map 表中。5. 用户应用软件应根据需要，通过 ECC_ENABLE 寄存器为 Flash 读取/取指路径启用 ECC 检查。Flash API 不会启用或禁用此寄存器。注意：ECC_ENABLE 在上电时默认启用。因此，用户应用软件应定义 NMI ISR 和 Flash 单比特错误 ISR，以便响应单比特和不可纠正的 Flash 错误。有关 FLASH_ECC_REGS 寄存器的详细信息，请参见器件专用的技术参考手册。

6. 如果用户应用的安全标准要求定期读取整个 Flash 存储器范围，以确保数据完整无损，则可执行此操作，并为整个 Flash 存储器范围计算校验和/CRC。可使用 VCU CRC 模块完成此工作。有关 VCU CRC 模块的详细信息，请参见器件专用的技术参考手册。

7. 如果用户应用的安全标准要求定期读取 Flash 寄存器空间，以确认寄存器配置完好，则可读取该空间，并为 Flash 寄存器存储器范围计算校验和/CRC。8. 如果用户应用的安全标准要求在编程和擦除操作后通过 CPU 读取来校验 Flash 内容，则可分别使用 Fapi_doVerify() 和 Fapi_doBlankCheck() 完成校验。9. Flash API 不会配置或管理 watchdog。用户应用可根据需要配置并管理 watchdog；既可在 Flash API 函数调用之间按顺序执行，也可使用 CPU 定时器 ISR。10. 如果 RAM 或 Flash 中的 Flash API 代码因应用设计不当（例如堆栈溢出）而被覆盖，CPU 尝试执行 Flash API 函数时可能发生 ITRAP（非法指令陷阱）。因此，用户应用应定义 ITRAP ISR 来响应该事件。

Copyright © 2024 Texas Instruments Incorporated

A Flash 状态机命令

表 A-1. Flash 状态机命令

Command	Description	Enumeration Type	API Call(s)
Program Data	Used to program data to any valid Flash address	Fapi_ProgramData	Fapi_issueProgrammingCommand() Fapi_issueProgrammingCommandForEccAddresses() Fapi_issueAutoEcc512ProgrammingCommand() Fapi_issueDataOnly512ProgrammingCommand() Fapi_issueDataAndEcc512ProgrammingCommand() Fapi_issueEccOnly64ProgrammingCommand()
Erase Sector	Used to erase a Flash sector located by the specified address	Fapi_EraseSector	Fapi_issueAsyncCommandWithAddress()
Erase Bank	Used to erase a Flash bank	Fapi_EraseBank	Fapi_issueBankEraseCommand()
Clear Status	Clears the status register	Fapi_ClearStatus	Fapi_issueAsyncCommand()

Copyright © 2024 Texas Instruments Incorporated

B 类型定义、宏定义、枚举和结构体

B.1 类型定义

```
#if defined(__TMS320C28XX__)  typedef unsigned char boolean; typedef unsign
ed int uint8; /*This is 16 bits in C28x*/ typedef unsigned int uint16;  typedef unsig
ned long int uint32;  typedef unsigned long long int uint64;  #endif
```

B.2 宏定义

```
#if (defined(__TMS320C28xx__) && __TI_COMPILER_VERSION__ < 6004000) #if !defined(__GNUC__) #err
or “F65 Flash API requires GCC language extensions. Use the –gcc option.” #endif #endif #ifndef TRUE    #defi
ne TRUE            1 #endif #ifndef FALSE    #define FALSE           0 #endif
```

B.3 枚举

B.3.1 Fapi_FlashProgrammingCommandsType

此类型包含 Fapi_IssueProgrammingCommand() 可使用的所有模式。

```
typedef enum {Fapi_AutoEccGeneration,   /* This is the default mode for the command and  will
auto generate the ecc for the provided data buffer */    Fapi_DataOnly,        /* Command will only process the data buffer *
/    Fapi_EccOnly,        /* Command will only process the ecc buffer */    Fapi_DataAndEcc       /* Command will process
data and ecc buffers */ }  ATTRIBUTE_PACKED Fapi_FlashProgrammingCommandsType;
```

B.3.2 Fapi_FlashBankType

用于指示当前使用的是哪个 Flash bank。

```
typedef enum {   Fapi_FlashBank0 }  ATTRIBUTE_PAC
KED Fapi_FlashBankType;
```

B.3.3 Fapi_FlashStateCommandsType

此类型包含所有可能的 Flash 状态机命令。

```
typedef enum {   Fapi_ProgramData    = 0x000
2,   Fapi_EraseSector    = 0x0006,   Fapi_Eras
eBank     = 0x0008,   Fapi_ClearStatus    = 0x
0010,

}  ATTRIBUTE_PACKED Fapi_FlashStateCommandsType;
```


Copyright © 2024 Texas Instruments Incorporated

B.3.4 Fapi_StatusType

这是包含所有可能返回状态代码的主类型。

```
typedef enum
{
   Fapi_Status_Success=0,           /* Function completed successfully */
   Fapi_Status_FsmBusy,             /* FSM is Busy */
   Fapi_Status_FsmReady,            /* FSM is Ready */
   Fapi_Status_AsyncBusy,           /* Async function operation is Busy */
   Fapi_Status_AsyncComplete,       /* Async function operation is Complete */
   Fapi_Error_Fail=500,             /* Generic Function Fail code */
   Fapi_Error_StateMachineTimeout,  /* State machine polling never returned ready and timed out */
   Fapi_Error_OtpChecksumMismatch,  /* Returned if OTP checksum does not match expected value */
   Fapi_Error_InvalidDelayValue,    /* Returned if the Calculated RWAIT value exceeds 15  -
                                       Legacy Error */
   Fapi_Error_InvalidHclkValue,     /* Returned if FClk is above max FClk value -
                                       FClk is a calculated from SYSCLK and RWAIT */
   Fapi_Error_InvalidCpu,           /* Returned if the specified Cpu does not exist */
   Fapi_Error_InvalidBank,          /* Returned if the specified bank does not exist */
   Fapi_Error_InvalidAddress,       /* Returned if the specified Address does not exist in Flash
                                       or OTP */
   Fapi_Error_InvalidReadMode,      /* Returned if the specified read mode does not exist */
   Fapi_Error_AsyncIncorrectDataBufferLength,
   Fapi_Error_AsyncIncorrectEccBufferLength,
   Fapi_Error_AsyncDataEccBufferLengthMismatch,
   Fapi_Error_FeatureNotAvailable,  /* Flash Wrapper feature is not available on this device */
   Fapi_Error_FlashRegsNotWritable, /* Returned if Flash registers are not writable due to
                                       security */
   Fapi_Error_InvalidCPUID,          /* Returned if OTP has an invalid CPUID */
}  ATTRIBUTE_PACKED Fapi_StatusType;
```

B.3.5 Fapi_ApiProductionStatusType

此结构用于在需要更灵活处理时返回状态值。

typedef enum { Alpha_Internal, /* For internal TI use only. Not intended to be used by customers */ Alpha, /* Early Engineering release. May not be functionally complete */ Beta_Internal, /* For internal TI use only. Not intended to be used by customers */ Beta, /* Functionally complete, to be used for testing and validation */ Production /* Fully validated, functionally complete, ready for production use */ } ATTRIBUTE_PACKED Fapi_ApiProductionStatusType;

B.4 结构体

B.4.1 Fapi_FlashStatusWordType

此结构用于在需要更灵活处理时返回状态值。

typedef struct { uint32 au32StatusWord[4]; } ATTRIBUTE_PACKED Fapi_FlashStatusWordType;

B.4.2 Fapi_LibraryInfoType

此结构用于返回 API 信息。

typedef struct { uint8 u8ApiMajorVersion; uint8 u8ApiMinorVersion; uint8 u8ApiRevision; Fapi_ApiProductionStatusType oApiProductionStatus; uint32 u32ApiBuildNumber; uint8 u8ApiTechnologyType; uint8 u8ApiTechnologyRevision; uint8 u8ApiEndianness; uint32 u32ApiCompilerVersion; } Fapi_LibraryInfoType;

Copyright © 2024 Texas Instruments Incorporated

参考资料

- 德州仪器：*TMS320F28P55x Real-Time Microcontrollers Data Manual (SPRSP85)*
- 德州仪器：*TMS320F28P55x Real-Time Microcontrollers Technical Reference Manual*

Copyright © 2024 Texas Instruments Incorporated

重要声明与免责声明

TI 按“原样”提供技术和可靠性数据（包括数据手册）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并对其所有缺陷不承担责任；TI 不作任何明示或默示保证，包括但不限于适销性、适用于特定用途或不侵犯第三方知识产权的默示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您全权负责：(1) 为您的应用选择适当的 TI 产品；(2) 设计、验证和测试您的应用；以及 (3) 确保您的应用符合适用标准及其他安全、信息安全、法规或其他要求。

这些资源如有更改，恕不另行通知。TI 仅允许您将这些资源用于开发使用资源中所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予您使用任何其他 TI 知识产权或第三方知识产权的许可。对于因您使用这些资源而产生的任何索赔、损害、费用、损失和责任，TI 概不负责；您须向 TI 及其代表提供充分赔偿，使其免受上述索赔、损害、费用、损失和责任的影响。

TI 产品的供应受 TI 销售条款或其他适用条款约束；相关条款可在 ti.com 上查阅，也可随 TI 产品一并提供。TI 提供这些资源不会扩大或以其他方式改变 TI 产品适用的保证或免责声明。

TI 反对并拒绝您可能提出的任何附加条款或不同条款。 重要声明

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2024, Texas Instruments Incorporated